Week 6 – Machine Learning and NLP: Text Classification

Workflow: **problem → text data → tokenization → numerical representation → model → evaluation → interpretation**

Tuesday – From text to features

- **NLP** means using computers to work with human language, e.g. spam detection, sentiment analysis, translation, summarization and question answering. This week the task is: *given a text, predict a category*.
- **Tokens and tokenization (Jurafsky & Martin ch. 2):** text is split into units (words, word parts, punctuation). This is a modelling choice and is not always as simple as splitting at spaces. Modern models use subword tokens (e.g. BPE).
- **Vocabulary:** all tokens learned from the training texts.
- **Bag of words:** each text becomes a vector of token counts. It is simple, fast and a strong baseline, but word order and context are mostly lost, and similar words count as different features.
- **CountVectorizer** stores raw counts. **TfidfVectorizer** gives more weight to terms that are frequent in a document but rare across all documents.

Short practical check

In [1]:
```
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer

texts = ["quiz upload failed", "project meeting tomorrow", "quiz result missing", "project group meeting"]
cv = CountVectorizer()
Xc = cv.fit_transform(texts)
display(pd.DataFrame(Xc.toarray(), columns=cv.get_feature_names_out()))

tv = TfidfVectorizer()
Xt = tv.fit_transform(texts)
pd.DataFrame(Xt.toarray(), columns=tv.get_feature_names_out()).round(2)
```

	failed	group	meeting	missing	project	quiz	result	tomorrow	upload
0	1	0	0	0	0	1	0	0	1
1	0	0	1	0	1	0	0	1	0
2	0	0	0	1	0	1	1	0	0
3	0	1	1	0	1	0	0	0	0

	failed	group	meeting	missing	project	quiz	result	tomorrow	upload
0	0.62	0.00	0.00	0.00	0.00	0.49	0.00	0.00	0.62
1	0.00	0.00	0.53	0.00	0.53	0.00	0.00	0.67	0.00
2	0.00	0.00	0.00	0.62	0.00	0.49	0.62	0.00	0.00
3	0.00	0.67	0.53	0.00	0.53	0.00	0.00	0.00	0.00

1. **One row** = one text (document).
2. **One column** = one vocabulary term.
3. **Many columns:** every different word becomes a column, so real text collections have thousands of columns (a sparse matrix, mostly zeros).
4. **Lost information:** word order, grammar and context, e.g. "upload failed" vs "failed upload" look the same.
5. **TF-IDF vs counts:** TF-IDF gives decimal weights. Words that appear in several texts (e.g. *quiz*, *project*, *meeting*) get lower weights than words found in only one text (e.g. *failed*, *tomorrow*).

Wednesday – Part A: Understand the data

In [2]:
```
df = pd.read_csv("synthetic-nlp-dataset.csv")
print(df.shape)
print("Missing values:", df.isna().sum().sum(), "| Duplicate texts:", df["message_text"].duplicated().sum())
display(df.head(3))
df["message_category"].value_counts()
```

```
(250, 6)
Missing values: 0 | Duplicate texts: 40
```

	message_id	student_key	course_week	channel	message_text	message_category
0	MSG0001	SYN023	2	course_chat	The result looks strange to me and I do not kn...	submission_deadline
1	MSG0002	SYN027	3	help_request	I have a question about the notebook; I want t...	assessment_feedback
2	MSG0003	SYN038	7	help_request	I have a question about Edge-LMS; I want to co...	submission_deadline

```
message_category
submission_deadline    50
assessment_feedback    50
project_group          50
course_content         50
technical_problem      50
Name: count, dtype: int64
```

In [3]:
```
pd.set_option("display.max_colwidth", 150)
df.groupby("message_category").head(2).sort_values("message_category")[["message_category", "message_text"]]
```

	message_category	message_text
1	assessment_feedback	I have a question about the notebook; I want to understand the feedback before I try again. I checked the instructions first. Also, I do not under...
17	assessment_feedback	I am stuck with this week's task and need some clarification.
5	course_content	Could you check the upload; I would like another example before continuing. I am trying to finish this before Monday.?
6	course_content	Something came up with the weekly task; the formula makes sense but the interpretation does not. I checked the instructions first.
3	project_group	I have a question about the project and what I should do next. we are trying to combine everyone's work into one result
4	project_group	I have a question about Edge-LMS; one member cannot attend and we need to reorganize the task. I checked the instructions first.
0	submission_deadline	The result looks strange to me and I do not know what to change.
2	submission_deadline	I have a question about Edge-LMS; I want to confirm when the final version has to be returned. I tried once more before asking.
12	technical_problem	I am not sure about the weekly task; I cannot tell whether the file actually reached the system. This is for this week's work.
22	technical_problem	I am stuck with this week's task and need some clarification. it worked earlier but now I cannot access it

1. **One row** = one synthetic course message.
2. **Raw text:** `message_text`.
3. **Target:** `message_category`.
4. **Categories:** assessment_feedback, course_content, project_group, submission_deadline, technical_problem.
5. **Class sizes:** 50 each, so the classes are perfectly balanced.
6. **Missing text:** none.
7. **Duplicates:** 40 repeated texts. Some generic messages (e.g. "*I am stuck with this week's task and need some clarification.*") appear up to 8 times, sometimes with **different** labels.
8. **Identifiers / context:** `message_id`, `student_key`, `course_week`, `channel`. These are not used as features.

Could a human always classify every message? No. Many messages are generic or mix two topics (e.g. an upload problem *and* a deadline question), so some ambiguity is expected.

Part B: Split the data

In [4]:
```
from sklearn.model_selection import train_test_split
X, y = df["message_text"], df["message_category"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, stratify=y, random_state=42)
print("train:", len(X_train), " test:", len(X_test))
```

```
train: 187  test: 63
```

The vectorizer is put inside a pipeline, so the vocabulary and TF-IDF weights are learned from the training texts only.

Part C: Baseline

In [5]:
```
from sklearn.dummy import DummyClassifier
from sklearn.metrics import accuracy_score
dummy = DummyClassifier(strategy="most_frequent").fit(X_train, y_train)
print("Baseline accuracy:", round(accuracy_score(y_test, dummy.predict(X_test)), 3))
```

```
Baseline accuracy: 0.19
```

Without reading the text, the accuracy is only about 0.19, because with 5 balanced classes guessing one class gets about 1 in 5 right.

Part D: TF-IDF + Logistic Regression

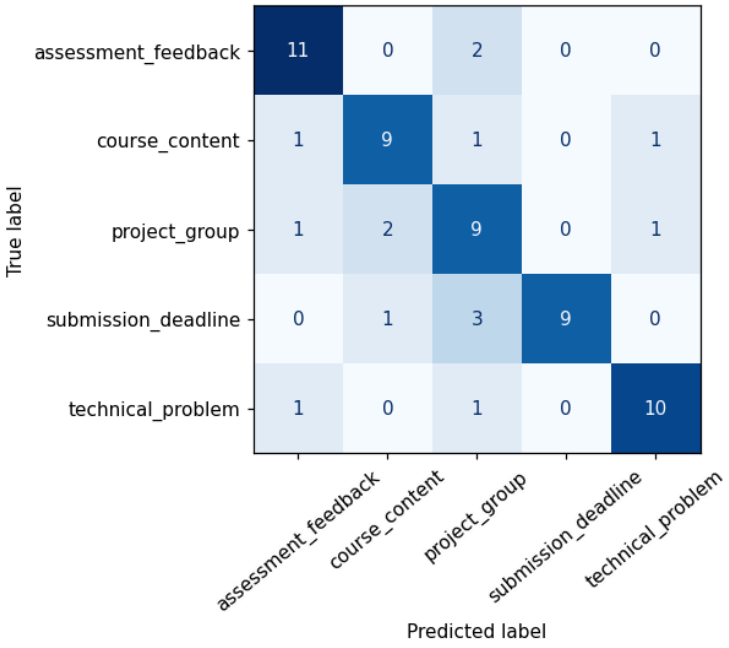
In [6]:
```
from sklearn.pipeline import make_pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

model = make_pipeline(TfidfVectorizer(), LogisticRegression(max_iter=1000))
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Accuracy:", round(accuracy_score(y_test, y_pred), 3))
print(classification_report(y_test, y_pred))
fig, ax = plt.subplots(figsize=(6, 5))
ConfusionMatrixDisplay.from_predictions(y_test, y_pred, ax=ax, xticks_rotation=40, colorbar=False, cmap="Blues")
plt.tight_layout(); plt.show()
```

```
Accuracy: 0.762
                    precision    recall  f1-score   support

assessment_feedback       0.79      0.85      0.81        13
     course_content       0.75      0.75      0.75        12
      project_group       0.56      0.69      0.62        13
submission_deadline       1.00      0.69      0.82        13
  technical_problem       0.83      0.83      0.83        12

           accuracy                           0.76        63
          macro avg       0.79      0.76      0.77        63
       weighted avg       0.79      0.76      0.77        63
```



Accuracy is about 0.76, which is far above the baseline (0.19). The most common confusion is **submission_deadline → project_group** (3 messages).

Part E: Multinomial Naive Bayes

VanderPlas explains that text gives many count-like features, and Multinomial Naive Bayes models these counts per class. It is fast and works well with TF-IDF or counts.

In [7]:
```
from sklearn.naive_bayes import MultinomialNB
nb = make_pipeline(TfidfVectorizer(), MultinomialNB()).fit(X_train, y_train)
print("TF-IDF + MultinomialNB accuracy:", round(accuracy_score(y_test, nb.predict(X_test)), 3))
```

```
TF-IDF + MultinomialNB accuracy: 0.762
```

Part F: Count vs TF-IDF

In [8]:
```
rows = []
for rep_name, vec in [("Count", CountVectorizer), ("TF-IDF", TfidfVectorizer)]:
    for clf_name, clf in [("LogisticRegression", LogisticRegression(max_iter=1000)), ("MultinomialNB",
MultinomialNB())]:
        p = make_pipeline(vec(), clf).fit(X_train, y_train).predict(X_test)
        rows.append({"Representation": rep_name, "Classifier": clf_name, "Accuracy": round(accuracy_score(y_test,
p), 3)})
pd.DataFrame(rows)
```

	Representation	Classifier	Accuracy
0	Count	LogisticRegression	0.841
1	Count	MultinomialNB	0.762
2	TF-IDF	LogisticRegression	0.762
3	TF-IDF	MultinomialNB	0.762

Representation	Classifier	Main observation
Count	Logistic Regression	best result here (0.84)
TF-IDF	Logistic Regression	0.76
Count / TF-IDF	Naive Bayes	0.76 for both

Did the representation matter? A little for Logistic Regression (a difference of 5 test messages) and not at all for Naive Bayes. With only 63 test messages, one message is about 1.6 %, so this difference is not very reliable.

Part G: Inspect mistakes

In [9]:
```
errors = pd.DataFrame({"message": X_test, "actual": y_test, "predicted": y_pred})
errors = errors[errors["actual"] != errors["predicted"]]
print("Number of errors:", len(errors))
errors
```

```
Number of errors: 15
```

	message	actual	predicted
20	I have a question about the notebook; I want to understand when this method is appropriate. I may have misunderstood something. Also, the page sto...	course_content	technical_problem
63	I have a question about the project and what I should do next. I cannot see which part of my answer caused the lost points	assessment_feedback	project_group
179	I am working on the group task; two people in our team are working on the same thing. I may have misunderstood something.	project_group	course_content
237	I am not sure about the upload; I am checking whether Monday morning is already too late. I may have misunderstood something. Also, I am unsure wh...	submission_deadline	course_content
197	Could you check the project; I am unsure why we split the data this way. Could you clarify it? Also, we need to divide the work before the next me...	course_content	project_group
111	I have a question about the project and what I should do next.	assessment_feedback	project_group
32	I am working on the group task; two people in our team are working on the same thing. I may have misunderstood something.	project_group	course_content
45	I have a question about the project and what I should do next.	submission_deadline	project_group
204	I'm working on the upload; two people in our team are working on the same thing. I checked the instructions first.	project_group	technical_problem
159	I am stuck with this week's task and need some clarification.	submission_deadline	project_group
189	I'm not sure about the upload; it worked earlier but now I cannot access it. Could you clarify it? Also, we need to divide the work before the nex...	technical_problem	project_group
171	Can you check the notebook? I am not sure what happened.	technical_problem	assessment_feedback
38	I am not sure about the result; two people in our team are working on the same thing. I checked the instructions first. Also, I am not sure how th...	project_group	assessment_feedback
141	I am stuck with this week's task and need some clarification.	submission_deadline	project_group
233	I NEED HELP WITH THE PROJECT; I WOULD LIKE ANOTHER EXAMPLE BEFORE CONTINUING. I CHECKED THE INSTRUCTIONS FIRST. ALSO, I WANT TO UNDERSTAND THE FEE...	course_content	assessment_feedback

Five errors examined:

Message (short)	Actual → predicted	Why it is difficult
"I have a question about the project and what I should do next."	assessment_feedback → project_group	a very short, generic message. The word *project* points to project_group, and the label is debatable
"I am stuck with this week's task and need some clarification."	submission_deadline → project_group	a duplicate generic text that appears with different labels in the data, so no model can be sure
"...the notebook... when this method is appropriate... the page stops loading"	course_content → technical_problem	mixes two topics. *page stops loading* is a technical phrase
"I'm not sure about the upload; it worked earlier... we need to divide the work..."	technical_problem → project_group	mixes two topics. *divide the work* is a strong project word
"I NEED HELP WITH THE PROJECT ... UNDERSTAND THE FEEDBACK ..."	course_content → assessment_feedback	all capitals and *feedback* is a misleading word here

Conclusion: the difficult messages are **short generic ones**, **messages that mix two topics**, and ones where one strong word (*project*, *feedback*, *upload*) points to the wrong class.

Thursday – 1. Evaluation

Because the classes are balanced, accuracy is a reasonable summary here, but the confusion matrix shows that **project_group** gets too many predictions: messages from other classes that mention *project* or *team* are pulled there. Precision for project_group is therefore lower than for the other classes.

2. N-grams

In [10]:
```
for ngr in [(1, 1), (1, 2)]:
    m = make_pipeline(TfidfVectorizer(ngram_range=ngr), LogisticRegression(max_iter=1000)).fit(X_train, y_train)
    vocab = len(m.named_steps["tfidfvectorizer"].vocabulary_)
    print(f"ngram_range={ngr}: vocabulary {vocab}, accuracy {accuracy_score(y_test, m.predict(X_test)):.3f}")
```

```
ngram_range=(1, 1): vocabulary 226, accuracy 0.762
ngram_range=(1, 2): vocabulary 840, accuracy 0.762
```

Adding bigrams makes the vocabulary much larger (more than 3× bigger), but the accuracy does not change. Expressions like *upload failed* or *group task* could help, but with this little data the extra features mostly add noise.

3. Preprocessing choice: lowercase

In [11]:
```
for low in [True, False]:
    m = make_pipeline(TfidfVectorizer(lowercase=low), LogisticRegression(max_iter=1000)).fit(X_train, y_train)
    print(f"lowercase={low}: accuracy {accuracy_score(y_test, m.predict(X_test)):.3f}")
```

```
lowercase=True: accuracy 0.762
lowercase=False: accuracy 0.730
```

Keeping upper case lowers accuracy (0.76 → 0.73), because *PROJECT* and *project* become different words. Some messages in the data are written in capitals, so lowercasing is useful here. It removes information about capitals, which does not matter for this task.

4. Vocabulary and interpretation

In [12]:
```
vec = model.named_steps["tfidfvectorizer"]
clf = model.named_steps["logisticregression"]
terms = vec.get_feature_names_out()
print("Vocabulary size:", len(terms), "| examples:", list(terms[:12]))
import numpy as np
for cls, coef in zip(clf.classes_, clf.coef_):
    print(f"{cls:20s}", ", ".join(terms[np.argsort(coef)[-6:][::-1]]))
```

```
Vocabulary size: 226 | examples: ['about', 'accepted', 'access', 'action', 'actual', 'actually', 'after', 'again',
'agreed', 'already', 'also', 'am']
assessment_feedback  was, my, result, feedback, attempt, score
course_content       example, this, understand, is, method, unclear
project_group        we, should, are, work, meeting, divide
submission_deadline  date, version, whether, which, final, know
technical_problem    page, system, unexpected, screen, get, message
```

The top words mostly make sense, e.g. *divide*, *meeting*, *we* for project_group, *page*, *system*, *screen* for technical_problem, and *feedback*, *score* for assessment_feedback. Some are general words (*this*, *is*, *was*) that happen to be more common in one class. These coefficients show how the model uses words. They do not show the writer's intention or anything about the person.

5. Leakage in NLP

In [13]:
```
print("Test messages that also appear word-for-word in training:", X_test.isin(X_train).sum())

dedup = df.drop_duplicates("message_text")
Xtr2, Xte2, ytr2, yte2 = train_test_split(dedup["message_text"], dedup["message_category"], test_size=0.25,
                                          stratify=dedup["message_category"], random_state=42)
m = make_pipeline(TfidfVectorizer(), LogisticRegression(max_iter=1000)).fit(Xtr2, ytr2)
print("After removing duplicate texts:", len(dedup), "messages, accuracy", round(accuracy_score(yte2,
m.predict(Xte2)), 3))
```

```
Test messages that also appear word-for-word in training: 13
After removing duplicate texts: 210 messages, accuracy 0.925
```

- The vectorizer is fitted only on training text (pipeline), so the test vocabulary does not leak.
- **Duplicate texts are a leakage risk:** 13 of 63 test messages also appear word-for-word in the training set. The model may simply "remember" them.
- After removing the duplicate texts, accuracy **rises** to about 0.93. Here most duplicates are short generic messages that appear with *different* labels, so they confuse the model instead of inflating the score. (The test set is also different and smaller, so the numbers are not directly comparable.)

- In a real project, identical or near-identical texts should be kept on the same side of the split.
- No column contains the category name, and no text is written after the label.

6. Privacy and ethics

Real messages could contain names, personal situations, health or family information, or information about other people. Before using real text, you need a clear purpose, only the parts of the text that are needed, limited access and storage time, and care that the model is not used to label students. This practical uses synthetic messages only, and the classifier says nothing about a student's personality, motivation or ability.

7. Reflection

1. **TF-IDF keeps** which words occur, how often, and how distinctive they are.
2. **It loses** word order, context and meaning (synonyms become different features).
3. **Why it still works:** the categories have typical keywords (*upload*, *deadline*, *group*, *feedback*).
4. **What the model misunderstood:** generic or mixed messages, and single strong words used in another context.
5. **A very high score would be suspicious** if it came from duplicate texts in train and test, from a column that reveals the label, or from a vectorizer fitted on all the data.

Friday – Modern NLP (Jurafsky & Martin ch. 5, transformers)

- **Bag of words / TF-IDF:** sparse vectors based on the vocabulary.
- **Embeddings:** learned dense vectors, where similar words get similar vectors.
- **Transformers / pretrained language models:** token representations depend on context and are learned from large text collections.
- **Large language models:** large pretrained models that can do many language tasks by prompting or fine-tuning.

TF-IDF is not all of NLP, but it is a transparent baseline that shows the complete workflow.

Practical verification values

In [14]:
```
print("Documents:", len(df), "| Classes:", df["message_category"].nunique())
print("Class frequencies:", df["message_category"].value_counts().to_dict())
print("Train / test:", len(X_train), "/", len(X_test))
print("TF-IDF vocabulary size (training data):", len(vec.vocabulary_))
print("Baseline accuracy:", round(accuracy_score(y_test, dummy.predict(X_test)), 3))
print("TF-IDF + Logistic Regression accuracy:", round(accuracy_score(y_test, y_pred), 3))
```

```
Documents: 250 | Classes: 5
Class frequencies: {'submission_deadline': 50, 'assessment_feedback': 50, 'project_group': 50, 'course_content': 50,
'technical_problem': 50}
Train / test: 187 / 63
TF-IDF vocabulary size (training data): 226
Baseline accuracy: 0.19
TF-IDF + Logistic Regression accuracy: 0.762
```

Friday – Concept map

